

Compreendendo *contexto*

O guia do Cursor diz: trabalhar com IA é, antes de tudo, **administrar o contexto** que você dá a ela. Nesta aula: o que é a janela de contexto, o que entra nela (sozinho ou porque você pediu), como ler o anel de contexto, o que acontece quando ela enche e como manter o agente focado, preciso e barato.

Janela de contexto

System prompt · Tools · Rules

@ menções

Busca e Explore

Anel de contexto

Resumo automático

Boas práticas

Formação Claude Code 2026 · Baseado em [cursor.com](#) (Learn: Context, Agents, Understanding your codebase; Docs: Prompting agents, Search, Subagents, Agent Overview, Ignore file; Help: @ mentions and context, Side chats) consultados em outubro de 2026 · Telas: reprodução ilustrativa

O que é *contexto*

Contexto é tudo o que o modelo "vê" num pedido: uma longa lista que funciona como a **memória de trabalho** da conversa. Tudo o que entra e tudo o que o modelo responde vai para essa lista.

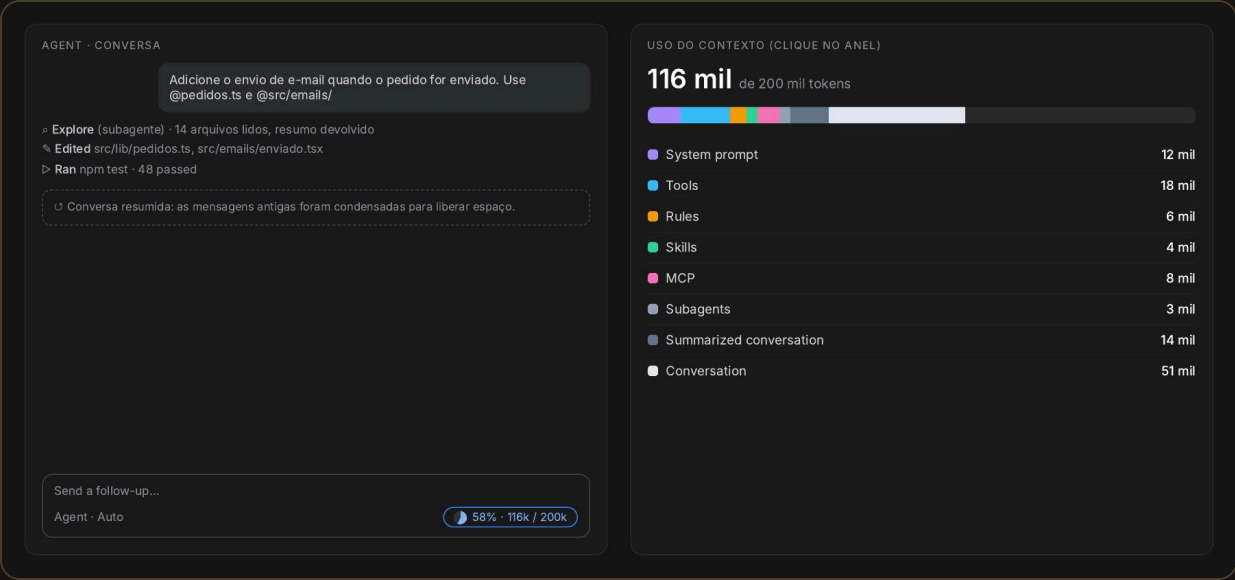
A ANALOGIA DO GUIA: UMA SOPA

Os **ingredientes** são o código, os arquivos e o seu pedido; a **receita** é o plano (às vezes sugerido pelo próprio modelo, com lista de tarefas); o **resultado** é o código gerado. Chefs diferentes, ou ingredientes diferentes, mudam o sabor.

- ✓ Modelos **não lembram** de uma conversa para outra; só sabem o que está no contexto
- ✓ O contexto **cresce** a cada mensagem, arquivo lido e resultado de ferramenta
- ✓ Cada modelo tem um **limite** (por exemplo, 200 mil ou 1 milhão de tokens)
- ✓ Tokens custam dinheiro: mais contexto = pedido mais caro

Melhorar o resultado é menos sobre escrever o prompt "perfeito" e mais sobre **dar o contexto certo**: nem de menos, nem de mais.

O que ocupa *a janela*



O anel de contexto e o detalhamento por categoria · reprodução ilustrativa, números de exemplo

System prompt · instruções internas do Cursor	Tools · definição de cada ferramenta do agente	Rules · rules do projeto e suas	Skills · descrições das skills
MCP · instruções e catálogo dos servidores	Subagents · tipos de subagente disponíveis	Summarized · resumo de turnos antigos	Conversation · mensagens, respostas e resultados

O que entra *sozinho*

Antes mesmo do seu pedido, o contexto já tem uma parte fixa, e o Cursor ainda acrescenta informações conforme o estado do projeto.

- **Fixo:** system prompt, definições das ferramentas, rules aplicáveis, descrições de skills e servidores MCP ligados.
- **Do projeto:** arquivos abertos, saída do terminal, erros do linter e o que o agente encontrar na busca.
- **Do trabalho:** cada arquivo lido, cada comando executado e cada resultado de ferramenta.

COMO O AGENTE BUSCA

Instant Grep acha nomes e textos exatos, com regex, mais rápido que o ripgrep em projetos grandes. Quando você não sabe o nome exato, o agente também **busca pelo significado** e combina as duas.

Privacidade: o índice de busca é montado **na sua máquina**. O Cursor não envia caminhos nem código para montar o índice e não guarda embeddings do código para a busca. Só o que o agente abre pode ir no pedido ao modelo.

O que entra *porque você pediu*

VOCÊ USA	ENTRA NO CONTEXTO
@auth.ts · @src/components/	Arquivos e pastas
@Terminals	A saída do terminal
@Chats	Uma conversa anterior
@Commit · @Branch	Mudanças não commitadas · diff do branch contra a main
@Browser	O navegador embutido
@nome-da-rule	Uma rule manual
Imagem colada ou arrastada	Print, mockup, mensagem de erro
Seleção + Ctrl + Shift + L	O trecho selecionado

Quando usar @ : quando você sabe quais arquivos importam (por exemplo, o componente e o teste dele). Quando não sabe, deixe o agente buscar sozinho.

Ao copiar código: Ctrl + V no chat cola como referência ao trecho; Ctrl + Shift + V cola como texto.

O que está no .cursorignore não entra pelo @ , nem pelo Agent, Tab ou Ctrl+K. Já o terminal e o MCP não são bloqueados por ele.

Quando a janela *enche*

Como numa conversa entre pessoas, fica difícil lembrar o que foi dito três horas atrás. Perto do limite, o Cursor **resume as partes antigas** da conversa para abrir espaço.

- ✓ O resumo aparece como "Summarized conversation" no detalhamento
- ✓ Detalhes antigos podem se perder no resumo
- ✓ No CLI, `/summarize` faz isso na hora que você quiser
- ✓ Trocar de modo (Agent, Ask, Plan, Debug) começa um contexto novo

SINAIS DE CONTEXTO CHEIO OU BAGUNÇADO

- O agente esquece uma decisão tomada no começo
- Repete abordagens que já falharam
- Mistura assuntos de tarefas diferentes
- Respostas mais lentas e mais caras

Remédio mais simples: abra uma conversa nova (`Ctrl` + `N`), com um resumo curto do que importa, ou retome com `@Chats`.

Recursos que *poupam contexto*

SUBAGENTE EXPLORE

Faz as buscas num **contexto próprio**, com um modelo mais rápido e várias buscas em paralelo, e devolve só o resumo. O agente usa sozinho quando a busca é ampla; você também pode pedir: *"use um subagente para achar todos os lugares onde validamos entrada do usuário"*.

SIDE CHATS

`/side` ou `/btw` abrem uma conversa lateral para uma dúvida paralela, sem encher a principal. Depois, `@` traz o que importar de volta.

RULES POR ARQUIVO

Rules com `globs` só entram quando aquele tipo de arquivo está em jogo, ao contrário das "Always Apply".

MCP COM MODERAÇÃO

Cada servidor ligado traz instruções e catálogo. Ligue só os que você usa no projeto.

.CURSORIGNORE

Pastas geradas e arquivos enormes fora da busca: menos ruído, menos tokens.

Perguntas que *usam bem o contexto*

ESPECÍFICA · VOCÊ SABE O NOME

Ache todos os arquivos que importam o `PaymentService` e mostre como tratam o `PaymentFailedError`.

O agente começa com busca exata (grep).

AMPLA · VOCÊ ESTÁ EXPLORANDO

Como o app trata pagamentos que falham? Mostre o fluxo do erro, do formulário até a mensagem na tela.

O agente começa buscando pelo significado.

ANTES DE MUDAR, ENTENDER

Antes de mudar qualquer coisa, me mostre como a validação de formulários funciona hoje e onde ficam os validadores compartilhados.

O erro mais comum, segundo o guia: pedir mudança sem contexto. O agente cria uma função que já existia ou segue um padrão diferente do resto do projeto.

Os agentes levam o pedido ao pé da letra: onde você não diz a intenção, eles decidem por conta própria.

Contexto *também é dinheiro*

Cada pedido envia o contexto da conversa ao modelo. Numa conversa longa, cada nova mensagem carrega tudo o que veio antes (ou o resumo dele). Agentes gastam bem mais tokens que perguntas simples, por causa das ferramentas e das tentativas.

MESMO PEDIDO, CONTEXTO DIFERENTE	EFEITO
Conversa nova, com  nos 2 arquivos certos	Rápido, focado, barato
Conversa de 3 horas sobre outros assuntos	Lento, caro, com risco de confusão

CACHE AJUDA

Vários modelos cobram bem menos pelo contexto repetido (leitura de cache): no Gemini 3.8 Flash, 90% de desconto. Por isso, manter as rules e o início da conversa estáveis também economiza.

Modelos com 1 milhão de tokens de contexto existem, mas encher a janela só porque cabe raramente melhora o resultado, e encarece.

Dez hábitos *de contexto*

- ✓ Uma conversa por tarefa
- ✓ Explore antes de mudar (Ask ou "não mude nada ainda")
- ✓ @ quando souber os arquivos; deixe o agente buscar quando não souber
- ✓ Diga a intenção, não só a ação
- ✓ Dúvidas paralelas em side chats
- ✓ Buscas amplas com o subagente Explore
- ✓ Rules curtas e por arquivo
- ✓ Só os servidores MCP que você usa
- ✓ `.cursorignore` para o que não ajuda
- ✓ De olho no anel de contexto; perto do limite, recomece

O que vale ser lembrado entre conversas vai para um **arquivo** (`AGENTS.md` , uma rule, um plano salvo no projeto), não para uma conversa que vai crescer sem fim.

Veja o contexto *na prática*

- 1 Abra uma conversa nova e clique no **anel de contexto**: veja quanto já está ocupado antes de você escrever.
- 2 Ligue um servidor MCP ou uma rule "Always Apply" e compare as categorias.
- 3 Peça um mapa amplo do projeto e veja o Explore trabalhando.
- 4 Faça uma pergunta específica com `@` em dois arquivos e compare o tamanho.
- 5 No painel de gastos, compare o custo dos dois pedidos.

O objetivo é criar o hábito de **olhar o anel** antes de pedir algo grande, do mesmo jeito que se olha o combustível antes de uma viagem.

Este exercício não precisa de um projeto grande: qualquer pasta com alguns arquivos de código serve.

Teste o que *ficou*

1. O que acontece quando a janela de contexto chega perto do limite?

A) O Cursor apaga o projeto

B) O Cursor resume as partes antigas da conversa

C) O modelo para de responder para sempre

D) Nada

B. E detalhes antigos podem se perder.

2. Para que serve o subagente Explore?

A) Instalar extensões

B) Fazer buscas num contexto próprio e devolver só o resumo

C) Trocar de modelo

D) Abrir o navegador

B. A conversa principal fica limpa.

3. O índice de busca do Cursor é montado onde?

A) Nos servidores do Cursor, com todo o código

B) Na sua máquina

C) No GitHub

D) No modelo

B. Só o que o agente abre vai no pedido.

4. Qual destes não ajuda a economizar contexto?

A) Conversa nova por tarefa

B) Rules por arquivo

C) Ligar todos os servidores MCP "por via das dúvidas"

D) Side chats para dúvidas paralelas

C. Cada servidor ocupa espaço.

Contexto certo, *resultado certo*

Contexto é a memória de trabalho do modelo: system prompt, ferramentas, rules, skills, MCP, a conversa e o que o agente leu. Parte entra sozinha, parte você traz com @. Acompanhe pelo **anel de contexto**, poupe espaço com o **Explore**, side chats, rules por arquivo e `.cursorignore`, e recomece a conversa quando ela crescer demais. Neste módulo você instalou, conheceu a tela, configurou, escolheu modelos (vendo quanto cada um custa) e rules, abriu projetos e, agora, aprendeu a cuidar do contexto.

Anel de contexto

@ · Ctrl+Shift+L

Explore · /side

Resumo automático

Conversa nova por tarefa

Fim do módulo 35 · Instalação e Configuração do Cursor

Fontes: cursor.com/learn (Context, Agents, Understanding your codebase); cursor.com/docs (Prompting agents, Search, Subagents, Agent Overview, Ignore file, Keyboard Shortcuts, Models & Pricing); cursor.com/help (@ mentions and context, Side chats), consultados em outubro de 2026. Os números da tela ilustrativa são exemplos.